



Computation & Complexity

Chap 4 Reducibility

Dr. Yuzhen Niu



4.1 Reducibility

- In this chapter we examine several additional unsolvable problems. In doing so we introduce the primary method for **proving that problems are computationally unsolvable**. It is called **reducibility**.
- A reduction is a way of **converting one problem to another problem** in such away that **a solution to the second problem can be used to solve the first problem**. Such reducibility comes up often in everyday life, even if we don't usually refer to them in this way.
- For example, suppose that you want to find your way around a new city. You know that doing so would be easy if you had a map. Thus you can reduce the problem of **finding your way around the city** to the problem of **obtaining a map of the city**.



Reducibility

- Reducibility always involves two problems, which we call A and B. **If A reduces to B, we can use a solution to B to solve A.** So in our example, A is the problem of finding your way around the city and B is the problem of obtaining a map. Note that reducibility says nothing about solving A or B alone, but only about the **solvability of A in the presence of a solution to B.**
- The following are further examples of reducibilities. The problem of traveling from Boston to Paris reduces to the problem of buying a plane ticket between the two cities. That problem in turn reduces to the problem of earning the money for the ticket. And that problem reduces to the problem of finding a job.



Reducibility

- Reducibility also occurs in **mathematical problems**. For example, the problem of measuring the area of a rectangle reduces to the problem of measuring its length and width. The problem of solving a system of linear equations reduces to the problem of inverting a matrix.
- Reducibility plays an important role in classifying problems by **decidability** and later in **complexity** theory as well. When A is reducible to B, **solving A cannot be harder than solving B** because a solution to B gives a solution to A. In terms of computability theory, **if A is reducible to B and B is decidable, A also is decidable**. Equivalently, **if A is undecidable and reducible to B, B is undecidable**. This last version is key to proving that various problems are undecidable.



Reducibility

- In short, our method for proving that a problem is undecidable will be to show that some other problem already known to be undecidable **reduces to it**.
- **$A \rightarrow$ reducible to B**
- **A is undecidable $\rightarrow B$ is undecidable**
- **B is decidable $\rightarrow A$ is decidable**



Reduce A_{TM} to $HALT_{TM}$

- We use the undecidability of A_{TM} to prove the undecidability of $HALT_{TM}$ by **reducing A_{TM} to $HALT_{TM}$** . Let
- $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ **accepts** } w \}$.
- $HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ **halts** on input } w \}$.
- **THEOREM $HALT_{TM}$ is undecidable.**



Reduce A_{TM} to $HALT_{TM}$

- PROOF IDEA

- This proof is by contradiction. We assume that $HALT_{TM}$ is decidable and use that assumption to show that A_{TM} is decidable, contradicting **Theorem A_{TM} is undecidable**. The key idea is to show that **A_{TM} is reducible to $HALT_{TM}$** . Let's assume that we have a TM R that decides $HALT_{TM}$. Then we use R to construct S , a TM that decides A_{TM} .
- To get a feel for the way to construct S , **pretend that you are S** . Your task is to decide A_{TM} . You are given an input of the form $\langle M, w \rangle$. You must output accept if M accepts w , and you must output reject if M loops or rejects on w . Try simulating M on w . If it accepts or rejects, do the same. But you may not be able to determine **whether M is looping**, and in that case your simulation will not terminate.



Reduce A_{TM} to $HALT_{TM}$

- That's bad, because **you are a decider and thus never permitted to loop**. So this idea, by itself, does not work. Instead, **use the assumption that you have TM R that decides $HALT_{TM}$** . With R, you can test whether M halts on w. If R indicates that M doesn't halt on w, reject because $\langle M, w \rangle$ isn't in A_{TM} . However, if R indicates that M does halt on w, you can do the simulation without any danger of looping.
- TM S: **先使用 $R(\langle M, w \rangle)$ 判断是否停机**, 不停机, 返回no; 停机, **计算 $M(w)$** , 返回 $M(w)$.
- Thus, if TM R exists, we can decide A_{TM} , but we know that A_{TM} is undecidable. By virtue of this contradiction we can conclude that R does not exist. Therefore $HALT_{TM}$ is undecidable.



Reduce A_{TM} to $HALT_{TM}$

- PROOF
- Let's **assume** for the purposes of obtaining a contradiction that **TM R decides $HALT_{TM}$** . We **construct TM S to decide A_{TM}** , with S operating as follows.
- S = "On input $\langle M, w \rangle$, an encoding of a TM M and a string w:
 - 1. **Run TM R on input $\langle M, w \rangle$** .
 - 2. If R rejects, reject.
 - 3. If R accepts, **simulate M on w until it halts**.
 - 4. If M has accepted, accept ; if M has rejected, reject."
- Clearly, if R decides $HALT_{TM}$, then S decides A_{TM} . Because A_{TM} is undecidable, $HALT_{TM}$ also must be undecidable.



4.2 Reductions Via Computation Histories

- The **computation history method** is an important technique for proving that A_{TM} is reducible to certain languages. This method is often useful when the problem to be shown undecidable **involves testing for the existence of something**. For example, this method is used to show the undecidability of Hilbert's tenth problem, testing for the existence of integral roots in a polynomial.
- The computation history for a Turing machine on an input is simply the **sequence of configurations** that the machine goes through as it processes the input. It is a complete record of the computation of this machine.



4.2 Reductions Via Computation Histories

- DEFINITION
- Let M be a Turing machine and w an input string. An **accepting computation history** for M on w is a sequence of configurations, $C_1, C_2, \dots, C_l$, where C_1 is the start configuration of M on w , C_l is an **accepting configuration** of M , and each C_i legally follows from C_{i-1} according to the rules of M . A **rejecting computation history** for M on w is defined similarly, except that C_l is a rejecting configuration.



4.2 Reductions Via Computation Histories

- Computation histories are finite sequences. If M doesn't halt on w , no accepting or rejecting computation history exists for M on w . **Deterministic machines have at most one computation history on any given input.** Nondeterministic machines may have many computation histories on a single input, corresponding to the various computation branches. For now, we continue to **focus on deterministic machines.** Our first undecidability proof using the computation history method concerns a type of machine called **a linear bounded automaton.**



4.2 Reductions Via Computation Histories

- DEFINITION
- A **linear bounded automaton** is a restricted type of Turing machine wherein **the tape head isn't permitted to move off the portion of the tape containing the input**. If the machine tries to move its head off either end of the input, the head stays where it is, in the same way that the head will not move off the left-hand end of an ordinary Turing machine's tape.

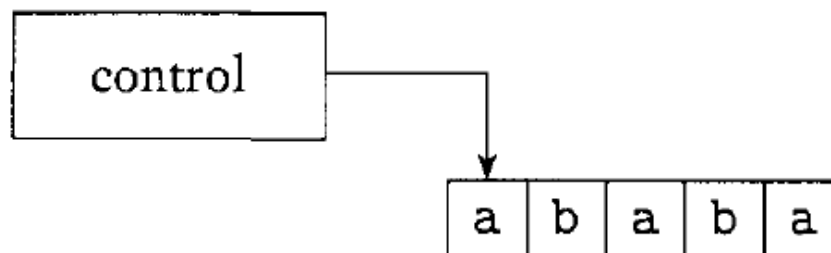


FIGURE 5.7

Schematic of a linear bounded automaton



4.2 Reductions Via Computation Histories

- A linear bounded automaton is a Turing machine with a limited amount of memory. It can only solve problems requiring memory that can fit within the tape used for the input. Using a tape alphabet larger than the input alphabet allows **the available memory to be increased up to a constant factor**. Hence we say that for an input of length n , the amount of memory available is linear in n ---thus the name of this model.



4.2 Reductions Via Computation Histories

- Let
- $A_{\text{LBA}} = \{ \langle M, w \rangle \mid M \text{ is an LBA that accepts input } w \}$.
- THEOREM
- A_{LBA} is decidable.



A_{LBA} is decidable

- LEMMA 5.8
 - Let M be an LBA with q states and g symbols in the tape alphabet. There are exactly qng^n distinct configurations of M for a tape of length n .
- PROOF
 - Recall that a configuration of M is like a snapshot in the middle of its computation. A configuration consists of **the state of the control, position of the head, and contents of the tape**. Here, M has q states. The length of its tape is n , so the head can be in one of n positions, and g^n possible strings of tape symbols appear on the tape. The product of these three quantities is the total number of different configurations of M with a tape of length n .



A_{LBA} is decidable

- A_{LBA} is decidable
- PROOF IDEA
 - In order to decide whether LBA M accepts input w , we simulate M on w . During the course of the simulation, if M halts and accepts or rejects, we accept or reject accordingly. The difficulty occurs **if M loops on w** . We need to be able to **detect looping** so that we can halt and reject.



A_{LBA} is decidable

- A_{LBA} is decidable
- PROOF IDEA
- The idea for **detecting when M is looping** is that, as M computes on w, it goes from configuration to configuration. If M ever repeats a configuration it would go on to repeat this configuration over and over again and thus be in a loop. Because M is an LBA, the amount of tape available to it is limited. By Lemma 5.8, M can be in only a limited number of configurations on this amount of tape. Therefore only a limited amount of time is available to M before it will enter some configuration that it has previously entered. **Detecting that M is looping is possible by simulating M for the number of steps given by Lemma 5.8. If M has not halted by then, it must be looping.**



A_{LBA} is decidable

- PROOF The algorithm that decides A_{LBA} is as follows.
- $L = \text{"On input } \langle M, w \rangle, \text{ where } M \text{ is an LBA and } w \text{ is a string:}$
 - 1. Simulate M on w for qng^n steps or until it halts.
 - 2. If M has halted, accept if it has accepted and reject if it has rejected. If it has not halted, reject."
- If M on w has not halted within qng^n steps, it must be **repeating a configuration** according to Lemma 5.8 and therefore looping. That is why our algorithm rejects in this instance.



4.3 A Simple Undecidable Problem

-Post Correspondence Problem, PCP

We can describe this problem easily as a type of puzzle. We begin with a collection of dominos, each containing two strings, one on each side. An individual domino looks like

$$\left[\begin{array}{c} a \\ \hline ab \end{array} \right]$$

and a collection of dominos looks like

$$\left\{ \left[\begin{array}{c} b \\ \hline ca \end{array} \right], \left[\begin{array}{c} a \\ \hline ab \end{array} \right], \left[\begin{array}{c} ca \\ \hline a \end{array} \right], \left[\begin{array}{c} abc \\ \hline c \end{array} \right] \right\}$$

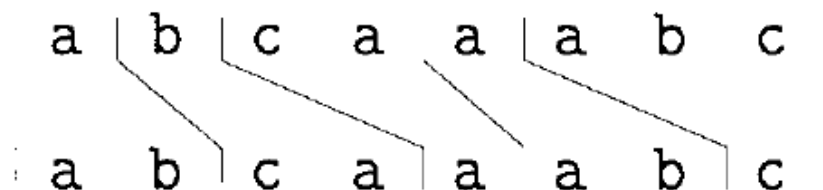
The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom. This list is called a *match*. For example, the following list is a match for this puzzle.



4.3 Post Correspondence Problem, PCP

$$\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right].$$

Reading off the top string we get $abcaaaabc$, which is the same as reading off the bottom. We can also depict this match by deforming the dominos so that the corresponding symbols from top and bottom line up.





4.3 Post Correspondence Problem, PCP

For some collections of dominos finding a match may not be possible. For example, the collection

$$\left\{ \left[\frac{abc}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{acc}{ba} \right] \right\}$$

cannot contain a match because every top string is longer than the corresponding bottom string.

The Post correspondence problem is to determine whether a collection of dominos has a match. This problem is unsolvable by algorithms.



4.3 Post Correspondence Problem, PCP

Before getting to the formal statement of this theorem and its proof, let's state the problem precisely and then express it as a language. An instance of the PCP is a collection P of dominos:

$$P = \left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\},$$

and a match is a sequence $i_1, i_2, \dots, i_l$, where $t_{i_1} t_{i_2} \cdots t_{i_l} = b_{i_1} b_{i_2} \cdots b_{i_l}$. The problem is to determine whether P has a match. Let

$$PCP = \{ \langle P \rangle \mid P \text{ is an instance of the Post correspondence problem with a match} \}.$$



4.3 Post Correspondence Problem, PCP

THEOREM 5.15 ..

$A_{TM} \rightarrow PCP$

PCP is undecidable.

PROOF IDEA Conceptually this proof is simple, though it involves many technical details. The main technique is reduction from A_{TM} via accepting computation histories. We show that from any TM M and input w we can construct an instance P where a match is an accepting computation history for M on w . If we could determine whether the instance has a match, we would be able to determine whether M accepts w .

How can we construct P so that a match is an accepting computation history for M on w ? We choose the dominos in P so that making a match forces a simulation of M to occur. In the match, each domino links a position or positions in one configuration with the corresponding one(s) in the next configuration.



4.3 Post Correspondence Problem, PCP

Before getting to the construction we handle three small technical points. (Don't worry about them too much on your initial reading through this construction.) First, for convenience in constructing P , we assume that M on w never attempts to move its head off the left-hand end of the tape. That requires first altering M to prevent this behavior. Second, if $w = \epsilon$, we use the string $\sqcup$ in place of w in the construction. Third, we modify the PCP to require that a match starts with the first domino,

$$\left[\begin{array}{c} t_1 \\ \hline b_1 \end{array} \right].$$

Later we show how to eliminate this requirement. We call this problem the modified Post correspondence problem (MPCP). Let



4.3 Post Correspondence Problem, PCP

$$A_{\text{TM}} \rightarrow \text{PCP}$$

$MPCP = \{\langle P \rangle \mid P \text{ is an instance of the Post correspondence problem with a match that starts with the first domino}\}.$

Now let's move into the details of the proof and design P to simulate M on w .

PROOF We let TM R decide the PCP and construct S deciding A_{TM} . Let

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where Q , Σ , Γ , and δ , are the state set, input alphabet, tape alphabet, and transition function of M , respectively.



DEFINITION 3.3

A *Turing machine* is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the *blank symbol* $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.



4.3 Post Correspondence Problem, PCP

$A_{TM} \rightarrow PCP$

$M \text{ accept } w?$

In this case S constructs an instance of the PCP P that has a match iff M accepts w . To do that S first constructs an instance P' of the MPCP. We describe the construction in seven parts, each of which accomplishes a particular aspect of simulating M on w . To explain what we are doing we interleave the construction with an example of the construction in action.

Part 1. The construction begins in the following manner.

q_0 : initial state

$w = w_1w_2...w_n$: input

Initial configuration

Put $\left[\frac{\#}{\#q_0w_1w_2 \cdots w_n\#} \right]$ into P' as the first domino $\left[\frac{t_1}{b_1} \right]$.

Because P' is an instance of the MPCP, the match must begin with this domino. Thus the bottom string begins correctly with $C_1 = q_0w_1w_2 \cdots w_n$, the first configuration in the accepting computation history for M on w , as shown in the following figure.



4.3 Post Correspondence

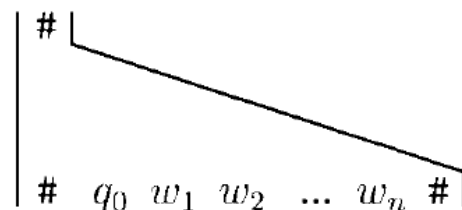


FIGURE 5.16

Beginning of the MPCP match

In this depiction of the partial match achieved so far, the bottom string consists of $\#q_0w_1w_2 \cdots w_n\#$ and the top string consists only of $\#$. To get a match we need to extend the top string to match the bottom string. We provide additional dominos to allow this extension. The additional dominos cause M 's next configuration to appear at the extension of the bottom string by forcing a single-step simulation of M .

In parts 2, 3, and 4, we add to P' dominos that perform the main part of the simulation. Part 2 handles head motions to the right, part 3 handles head motions to the left, and part 4 handles the tape cells not adjacent to the head.



4.3 Post Correspondence Problem, PCP

Part 2. For every $a, b \in \Gamma$ and every $q, r \in Q$ where $q \neq q_{\text{reject}}$,

if $\delta(q, a) = (r, b, R)$, put $\left[\frac{qa}{br} \right]$ into P' .

Part 3. For every $a, b, c \in \Gamma$ and every $q, r \in Q$ where $q \neq q_{\text{reject}}$,

if $\delta(q, a) = (r, b, L)$, put $\left[\frac{cqa}{rcb} \right]$ into P' .

Part 4. For every $a \in \Gamma$,

put $\left[\frac{a}{a} \right]$ into P' .



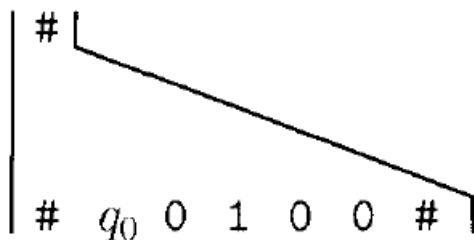
4.3 Post Correspondence Problem, PCP

Now we make up a hypothetical example to illustrate what we have built so far. Let $\Gamma = \{0, 1, 2, \sqcup\}$. Say that w is the string 0100 and that the start state of M is q_0 . In state q_0 , upon reading a 0, let's say that the transition function dictates that M enters state q_7 , writes a 2 on the tape, and moves its head to the right. In other words, $\delta(q_0, 0) = (q_7, 2, R)$.

Part 1 places the domino

$$\left[\frac{\#}{\#q_0 0100\#} \right] = \left[\frac{t_1}{b_1} \right]$$

in P' , and the match begins:





4.3 Post Correspondence Problem, PCP

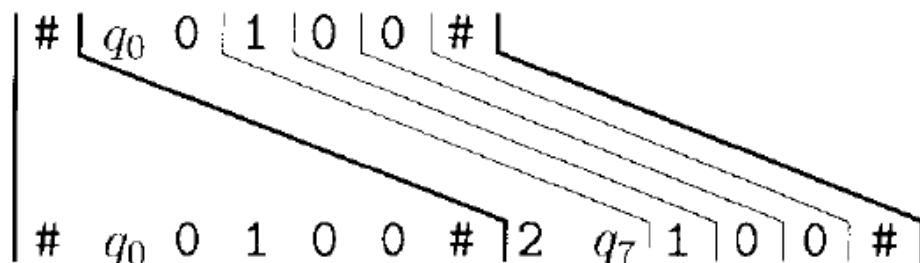
In addition, part 2 places the domino

$$\left[\begin{array}{c} q_0 0 \\ 2q_7 \end{array} \right]$$

as $\delta(q_0, 0) = (q_7, 2, R)$ and part 4 places the dominos

$$\left[\begin{array}{c} 0 \\ 0 \end{array} \right], \left[\begin{array}{c} 1 \\ 1 \end{array} \right], \left[\begin{array}{c} 2 \\ 2 \end{array} \right], \text{ and } \left[\begin{array}{c} \sqcup \\ \sqcup \end{array} \right]$$

in P' , as 0, 1, 2, and $\sqcup$ are the members of Γ . That, together with part 5, allows us to extend the match to





4.3 Post Correspondence Problem, PCP

Thus the dominos of parts 2, 3, and 4 let us extend the match by adding the second configuration after the first one. We want this process to continue, adding the third configuration, then the fourth, and so on. For it to happen we need to add one more domino for copying the # symbol.



4.3 Post Correspondence Problem, PCP

Part 5.

Put $\left[\begin{smallmatrix} \# \\ \# \end{smallmatrix} \right]$ and $\left[\begin{smallmatrix} \# \\ \sqcup \# \end{smallmatrix} \right]$ into P' .

The first of these dominos allows us to copy the $\#$ symbol that marks the separation of the configurations. In addition to that, the second domino allows us to add a blank symbol $\sqcup$ at the end of the configuration to simulate the infinitely many blanks to the right that are suppressed when we write the configuration.

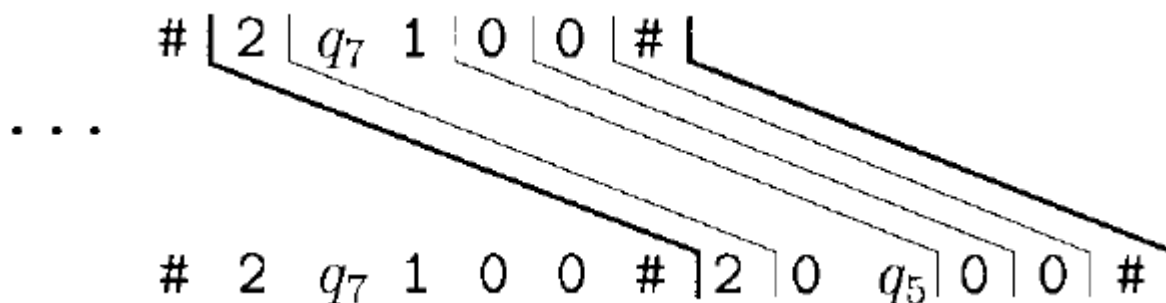


4.3 Post Correspondence Problem, PCP

Continuing with the example, let's say that in state q_7 , upon reading a 1, M goes to state q_5 , writes a 0, and moves the head to the right. That is, $\delta(q_7, 1) = (q_5, 0, R)$. Then we have the domino

$$\left[\begin{array}{c} q_7 1 \\ 0 q_5 \end{array} \right] \text{ in } P'.$$

So the latest partial match extends to




$$\left[\frac{0q_50}{q_902}\right], \left[\frac{1q_50}{q_912}\right], \left[\frac{2q_50}{q_922}\right], \text{ and } \left[\frac{\sqcup q_50}{q_9\sqcup2}\right].$$
$$\begin{array}{ccccccc} \# & | & 2 & | & 0 & q_5 & 0 \\ & \searrow & & & & & \\ \dots & & & & & & \\ & \searrow & & & & & \\ \# & & 2 & 0 & q_5 & 0 & 0 & \# & | & 2 & q_9 & 0 & 2 & 0 & \# \end{array}$$



4.3 Post Correspondence Problem, PCP

Note that, as we construct a match, we are forced to simulate M on input w . This process continues until M reaches a halting state. If an accept state occurs, we want to let the top of the partial match “catch up” with the bottom so that the match is complete. We can arrange for that to happen by adding additional dominos.

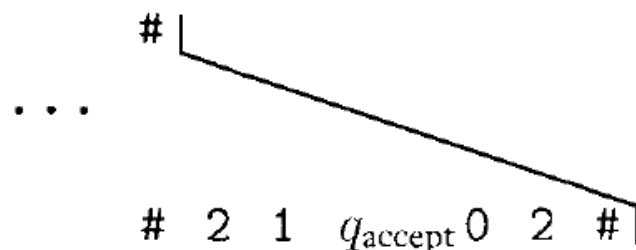


4.3 Post Correspondence Problem, PCP

Part 6. For every $a \in \Gamma$,

put $\left[\frac{a q_{\text{accept}}}{q_{\text{accept}}} \right]$ and $\left[\frac{q_{\text{accept}} a}{q_{\text{accept}}} \right]$ into P' .

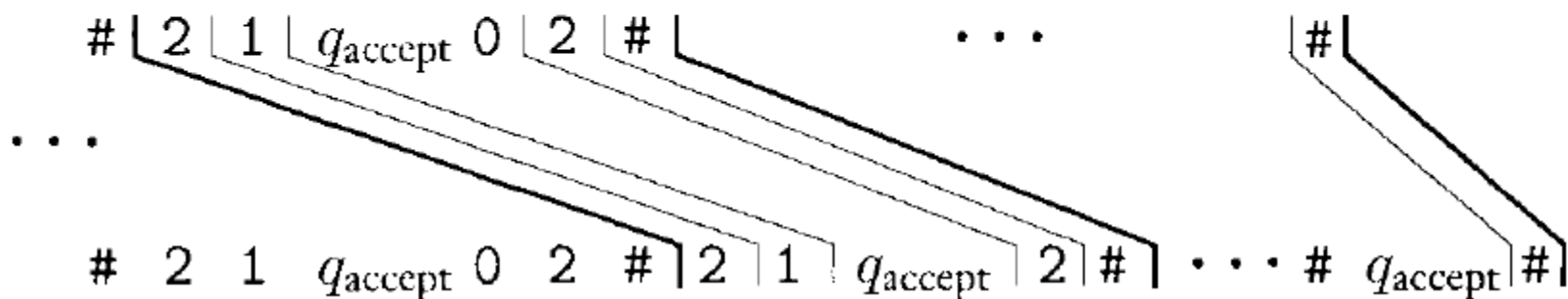
This step has the effect of adding “pseudo-steps” of the Turing machine after it has halted, where the head “eats” adjacent symbols until none are left. Continuing with the example, if the partial match up to the point when the machine halts in an accept state is





4.3 Post Correspondence Problem, PCP

The dominos we have just added allow the match to continue:





4.3 Post Correspondence Problem, PCP

Part 7. Finally we add the domino

$$\left[\frac{q_{\text{accept}} \# \#}{\#} \right]$$

and complete the match:

$$\begin{array}{c} \dots \\ \begin{array}{c} \# \mid q_{\text{accept}} \# \# \\ \diagdown \\ \# \mid q_{\text{accept}} \# \mid \# \end{array} \end{array}$$



4.3 Post Correspondence Problem, PCP

That concludes the construction of P' . Recall that P' is an instance of the MPCP whereby the match simulates the computation of M on w . To finish the proof, we recall that the MPCP differs from the PCP in that the match is required to start with the first domino in the list.



4.3 Post Correspondence Problem, PCP

We now show how to convert P' to P , an instance of the PCP that still simulates M on w . We do so with a somewhat technical trick. The idea is to build the requirement of starting with the first domino directly into the problem so that stating the explicit requirement becomes unnecessary. We need to introduce some notation for this purpose.

Let $u = u_1 u_2 \cdots u_n$ be any string of length n . Define $\star u$, $u\star$, and $\star u\star$ to be the three strings

$$\begin{aligned}\star u &= \star u_1 \star u_2 \star u_3 \star \cdots \star u_n \\ u\star &= u_1 \star u_2 \star u_3 \star \cdots \star u_n \star \\ \star u\star &= \star u_1 \star u_2 \star u_3 \star \cdots \star u_n \star .\end{aligned}$$

Here, $\star u$ adds the symbol $\star$ before every character in u , $u\star$ adds one after each character in u , and $\star u\star$ adds one both before and after each character in u .



4.3 Post Correspondence Problem, PCP

To convert P' to P , an instance of the PCP, we do the following. If P' were the collection

$$\left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \left[\frac{t_3}{b_3} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\},$$

we let P be the collection

$$\left\{ \left[\frac{\star t_1}{\star b_1 \star} \right], \left[\frac{\star t_1}{b_1 \star} \right], \left[\frac{\star t_2}{b_2 \star} \right], \left[\frac{\star t_3}{b_3 \star} \right], \dots, \left[\frac{\star t_k}{b_k \star} \right], \left[\frac{\star \diamond}{\diamond} \right] \right\}.$$



4.3 Post Correspondence Problem, PCP

Considering P as an instance of the PCP, we see that the only domino that could possibly start a match is the first one,

$$\left[\frac{\star t_1}{\star b_1 \star} \right],$$

because it is the only one where both the top and the bottom start with the same symbol—namely, $\star$. Besides forcing the match to start with the first domino, the presence of the $\star$ s doesn't affect possible matches because they simply interleave with the original symbols. The original symbols now occur in the even positions of the match. The domino

$$\left[\frac{\star \diamond}{\diamond} \right]$$

is there to allow the top to add the extra $\star$ at the end of the match.



4.4 Mapping Reducibility/Many-One Reducibility

- The notion of reducing one problem to another may be defined formally one of several ways. The choice of which one to use depends on the application. Our choice is a simple type of reducibility called **mapping reducibility**. It is called **many-one reducibility** in some other textbooks.
- Roughly speaking, being able to reduce problem A to problem B by using a mapping reducibility means that a **computable function** exists that converts instances of problem A to instances of problem B. If we have such a conversion function, called a reduction, we can **solve A with a solver for B**. The reason is that any instance of A can be solved by first using the reduction to convert it to an instance of B and then applying the solver for B. A precise definition of mapping reducibility follows shortly.



4.4 Mapping Reducibility/Many-One Reducibility

- COMPUTABLE FUNCTIONS
 - A Turing machine computes a function by starting with the input to the function on the tape and halting with the output of the function on the tape.
 - DEFINITION: A function $f: \Sigma^* \rightarrow \Sigma^*$ is a **computable function** if some Turing machine M , on every input w , halts with just **$f(w)$** on its tape.
 - For example a machine that takes input $\langle m, n \rangle$ and returns $m + n$, the sum of m and n .



4.4 Mapping Reducibility/Many-One Reducibility

- DEFINITION
- Language A is **mapping reducible** to language B , written $A \leq_m B$, if there is a computable function $f : \Sigma^* \rightarrow \Sigma^*$, where for every w ,
 $w \in A \Leftrightarrow f(w) \in B$.

The function f is called the **reduction** of A to B .

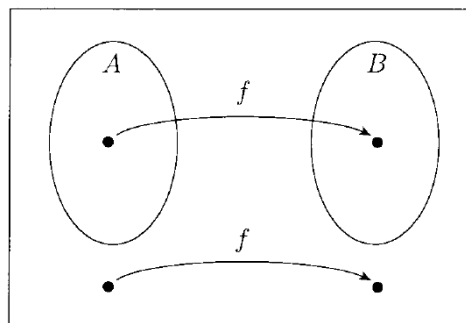


FIGURE 5.21
Function f reducing A to B



4.4 Mapping Reducibility/Many-One Reducibility

- THEOREM 5.22 : If $A \leq_m B$ and B is decidable, then A is decidable.
- PROOF:
 - We let M be the decider for B and f be the reduction from A to B . We describe a decider N for A as follows.
 - $N =$ "On input w :
 - 1. Compute $f(w)$.
 - 2. Run M on input $f(w)$ and output whatever M outputs."
- Clearly, if $w \in A$, then $f(w) \in B$ because f is a reduction from A to B . Thus M accepts $f(w)$ whenever $w \in A$. Therefore N works as desired.



4.4 Mapping Reducibility/Many-One Reducibility

- The following corollary of Theorem 5.22 has been our main tool for proving undecidability.
- COROLLARY 5.23
 - If $A \leq_m B$ and A is undecidable, then B is undecidable.
- Now we revisit some of our earlier proofs that used the reducibility method to get examples of mapping reducibilities.



4.4 Mapping Reducibility/Many-One Reducibility

- In Theorem 5.1 we used a reduction from A_{TM} to prove that $HALT_{TM}$ is undecidable. This reduction showed how a decider for $HALT_{TM}$ could be used to give a decider for A_{TM} . We can demonstrate a **mapping reducibility from A_{TM} to $HALT_{TM}$** as follows.
- To do so we must present a computable function f that takes input of the form $\langle M, w \rangle$ and returns output of the form $\langle M', w' \rangle$, where
- **$\langle M, w \rangle \in A_{TM}$ if and only if $\langle M', w' \rangle \in HALT_{TM}$.**
- The following machine F computes a reduction f .
 - $F =$ "On input $\langle M, w \rangle$:
 - 1. Construct the following machine M' .
 - $M' =$ "On input x :
 - 1. Run M on x .
 - 2. If M accepts, accept.
 - 3. If M rejects, enter a loop."
 - 2. Output $\langle M', w \rangle$."



4.4 Mapping Reducibility/Many-One Reducibility

- A minor issue arises here concerning improperly formed input strings. If TM F determines that its input is not of the correct form as specified in the input line “On input $\langle M, w \rangle$ ” and hence that the input is not in A_{TM} , the TM outputs a string not in $HALT_{TM}$. Any string not in $HALT_{TM}$ will do.
- In general, when we describe a Turing machine that computes a reduction from A to B , improperly formed inputs are assumed to map to strings outside of B .



4.4 Mapping Reducibility/Many-One Reducibility

- The proof of the undecidability of the Post correspondence problem in Theorem 5.15 contains two mapping reductions. First, it shows that $A_{TM} \leq_m \text{MPCP}$ and then it shows that $\text{MPCP} \leq_m \text{PCP}$. In both cases we can easily obtain the actual reduction function and show that it is a mapping reduction. Mapping reducibility is transitive, so these two reductions together imply that $A_{TM} \leq_m \text{PCP}$.



4.4 Mapping Reducibility/Many-One Reducibility

- THEOREM 5.28
- If $A \leq_m B$ and B is Turing-recognizable, then A is Turing-recognizable.
- The proof is the same as that of Theorem 5.22, except that M and N are recognizers instead of deciders.
- COROLLARY 5.29
- If $A \leq_m B$ and A is not Turing-recognizable, then B is not Turing-recognizable.



4.5 Undecidable Problems about Turing machines

•

conclude that *there is no algorithm that decides, for an arbitrary given Turing machine M and input string w , whether or not M accepts w .* Problems for which no algorithms exist are called **undecidable** or **unsolvable**; we shall see many of them in this chapter. The most famous and fundamental undecidable problem is the one of telling whether a given Turing machine halts on a given input —whose undecidability we just established. This problem is usually called **the halting problem for Turing machines.**



4.5 Undecidable Problems about Turing machines

Once we have established, by diagonalization, that the halting problem is undecidable, the undecidability of a great variety of problems follows. These results are proved not by further diagonalizations, but by *reductions*: we show in each case that if some language L_2 were recursive, then so would be some language L_1 , already known *not* to be recursive.

Definition 5.4.1: Let $L_1, L_2 \subseteq \Sigma^*$ be languages. A **reduction from L_1 to L_2** is a recursive function $\tau : \Sigma^* \mapsto \Sigma^*$ such that $x \in L_1$ if and only if $\tau(x) \in L_2$.



4.5 Undecidable Problems about Turing machines

The reader must take care to understand the “direction” in which a reduction is to be applied. To show that a language L_2 is not recursive, we must identify a language L_1 that is known to be not recursive, and then reduce L_1 to L_2 .

Theorem 5.4.1: *If L_1 is not recursive, and there is a reduction from L_1 to L_2 , then L_2 is also not recursive.*

Proof: Suppose that L_2 is recursive, say decided by Turing machine M_2 , and let T be the Turing machine that computes the reduction τ . Then the Turing machine TM_2 would decide L_1 . But L_1 is undecidable — a contradiction. ■

$A \rightarrow$ reducible to B

A is undecidable $\rightarrow B$ is undecidable

B is decidable $\rightarrow A$ is decidable



4.5 Undecidable Problems about Turing machines

We next use reductions to show that several problems about Turing machines are undecidable.

Theorem 5.4.2: *The following problems about Turing machines are undecidable.*

- (a) *Given a Turing machine M and an input string w , does M halt on input w ?*
- (b) *Given a Turing machine M , does M halt on the empty tape?*
- (c) *Given a Turing machine M , is there any string at all on which M halts?*
- (d) *Given a Turing machine M , does M halt on every input string?*
- (e) *Given two Turing machines M_1 and M_2 , do they halt on the same input strings?*



4.5 Undecidable Problems about Turing machines

(b) *Given a Turing machine M , does M halt on the empty tape?*

Proof: Part (a) was proved in the previous section.

(b) We describe a reduction from H to the language

$$L = \{ "M" : M \text{ halts on } e \}.$$

Given the description of a Turing machine M and an input x , our reduction simply constructs the description of a Turing machine M_w that operates as follows: M_w , when started on the empty tape (that is, in configuration $(s, \triangleright \sqcup)$), writes w on its tape and then starts to simulate M . In other words, if $w = a_1 \cdots a_n$, then M_w is simply the machine

$$Ra_1Ra_2R \dots Ra_nL_{\sqcup}M.$$

And it is easy to see that the function τ that maps " M " " w " to " M_w " is indeed recursive.



4.5 Undecidable Problems about Turing machines

- (c) *Given a Turing machine M , is there any string at all on which M halts?*
(d) *Given a Turing machine M , does M halt on every input string?*

(c) We can reduce the language L shown to be nonrecursive in Part (b) to the language $L' = \{“M” : M \text{ halts on some input}\}$, as follows. Given the representation of any Turing machine M , our reduction constructs the representation of a Turing machine M' that erases any input it is given, and then simulates M on the empty string. Clearly, M' halts on some string if and only if it halts on all strings, if and only if M halts on the empty string.

(d) The argument for Part (c) works here as well, since M' is constructed so that it accepts some input if and only if it accepts every input.



4.5 Undecidable Problems about Turing machines

- (e) *Given two Turing machines M_1 and M_2 , do they halt on the same input strings?*

(e) We shall reduce the problem in Part (d) to this one. Given the description of a machine M , our reduction constructs the string

$$\tau("M") = "M" "y",$$

where “ y ” is the description of the machine that immediately accepts any input. Clearly, the two machines M and y accept the same inputs if and only if M accepts all inputs.



4.6 Problems of Recursive Languages

We have already seen that every recursive language is recursively enumerable, but the two classes are not the same: The language H is one that bears witness to the difference. Which recursively enumerable languages are recursive? There are many ways to answer this question; we present one next.

$$H = \{ \text{"}M\text{"} \text{"}w\text{"} : \text{Turing machine } M \text{ halts on input string } w \}.$$

Notice first that H is recursively enumerable: It is precisely the language semidecided by our universal Turing machine U in the previous section. Indeed, on input " M " " w ", U halts precisely when the input is in H .



4.6 Problems of Recursive Languages

Theorem 5.7.1: *A language is recursive if and only if both it and its complement are recursively enumerable.*

Proof: If L is recursive, then L is recursively enumerable by Theorem 4.2.1; also, $\bar{L}$ is recursive, and hence recursively enumerable, by Theorem 4.2.2.

For the other direction, suppose that L is semidecided by M_1 and $\bar{L}$ is semidecided by M_2 . Then we can construct a Turing machine M that decides L . For convenience, we describe M as a 2-tape machine; by Theorem 4.3.1, M can be simulated by a 1-tape machine. The machine M begins by putting its input string w on both tapes and placing its heads at the right ends of both copies of the input. Then M simulates both M_1 and M_2 in parallel: at each step of the operation of M , one step of M_1 's computation is carried out on the first tape, and one step of M_2 's computation is carried out on the second tape. Since either M_1 or M_2 must halt on w , but not both, M eventually reaches a situation in which the simulated version of either M_1 or M_2 is about to halt. When this happens, M determines which of M_1 and M_2 was about to halt, and halts on y or n accordingly. ■



4.6 Problems of Recursive Languages

Definition 5.7.1: We say that a Turing machine M **enumerates** the language L if and only if, for some fixed state q of M ,

$$L = \{w : (s, \triangleright \sqcup) \vdash_M^* (q, \triangleright \sqcup w)\}.$$

A language is **Turing-enumerable** if and only if there is a Turing machine that enumerates it.

That is, M enumerates L by starting from blank tape and computing away, periodically passing through a special state q (*not* a halting state). Entering state q signals that the string currently on M 's tape is a member of L ; M may then leave state q and reenter it later on with some other member of L on its tape. Notice that the members of L can be listed in any order and with repetitions.



4.6 Problems of Recursive Languages

Theorem 5.7.2: *A language is recursively enumerable if and only if it is Turing-enumerable.*

Proof: Suppose that L is semidecided by Turing machine M . Then we can design a machine M' which, instead of taking a string as input, starts from the empty tape, systematically generates (for instance, in lexicographic order) all strings over the alphabet of L , and carries out on each the same computation that M would carry out.

The solution is based on a version of the the “dovetailing” procedure we used in Section 1.4 to show that a union of countably many sets is countable. Instead of attempting to complete the computation on each string as it is generated, M' carries out the following sequence of operations:



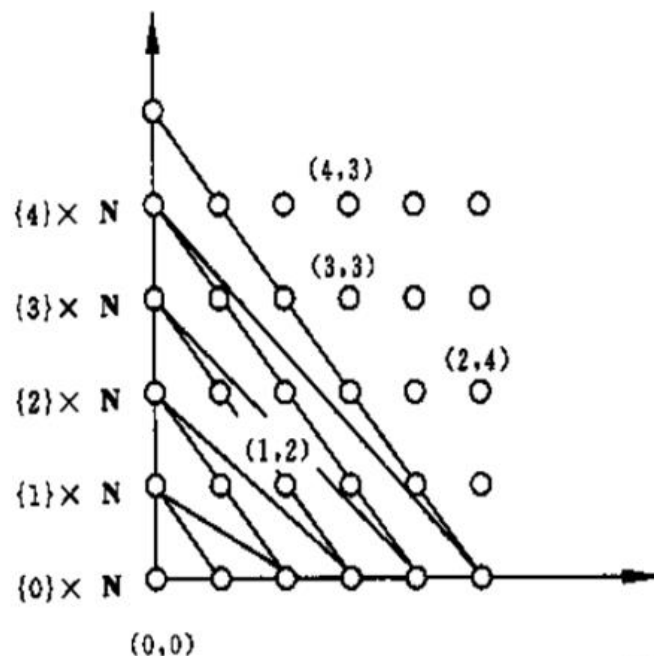
1.4 Finite and Infinite Sets

- The above ideas can be used to show that **the union of a countably infinite collection of countably infinite sets is countably infinite.**
- For example, let us show that $\mathbb{N} \times \mathbb{N}$ is countably infinite; note that $\mathbb{N} \times \mathbb{N}$ is the union of $\{0\} \times \mathbb{N}$, $\{1\} \times \mathbb{N}$, $\{2\} \times \mathbb{N}$, and so on, that is, the union of a countably infinite collection of countably infinite sets.
Dovetailing must here be more subtle than in the example above: we cannot, as we did there, visit one element from each set before visiting the second element of the first set. Instead we proceed as follows:



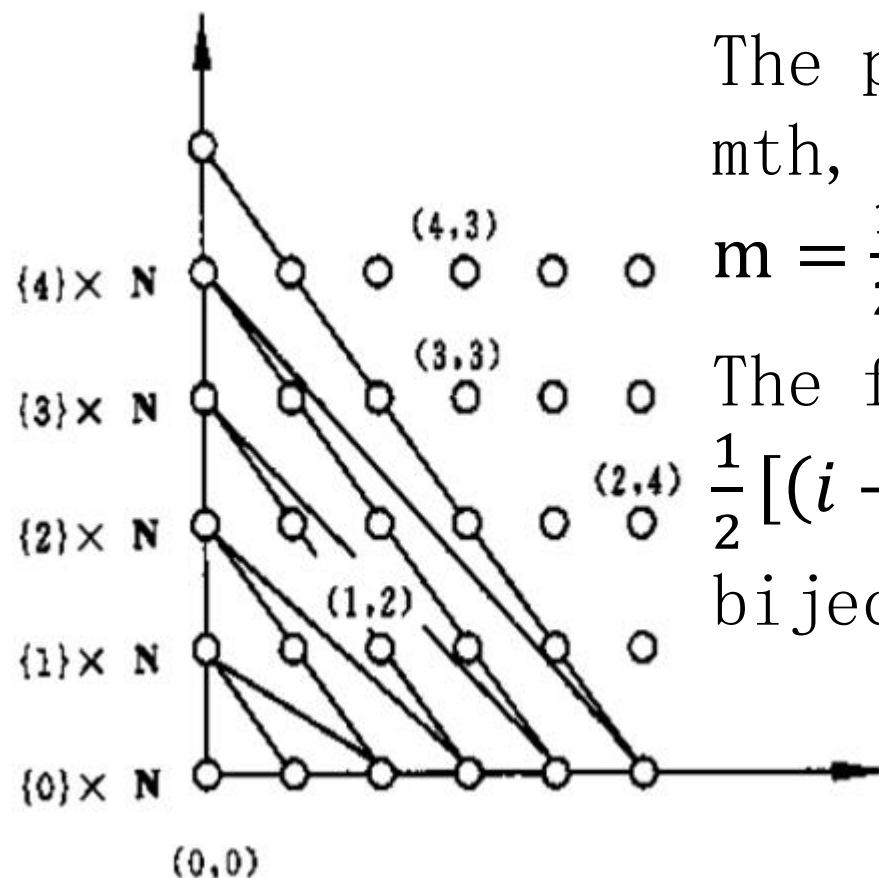
1.4 Finite and Infinite Sets

- (1) In the first round, we visit one element from the first set: $(0, 0)$.
- (2) In the second round, we visit the next element from the first set, $(0, 1)$, and also the first element from the second set, $(1, 0)$.
- (3) In the third round we visit the next unvisited elements of the first and second sets, $(0, 2)$ and $(1, 1)$, and also the first element of the third set, $(2, 0)$.
- (4) In general, in the n -th round, we visit the n -th element of the first set, the $(n - 1)$ -st element of the second set, and the first element of the n th set.





1.4 Finite and Infinite Sets



The pair (i, j) is visited m th, where

$$m = \frac{1}{2}[(i + j)^2 + 3i + j].$$

The function $f(i, j) = \frac{1}{2}[(i + j)^2 + 3i + j]$ is a bijection from $N \times N$ to N .



4.6 Problems of Recursive Languages

(Phase 1) First M' carries out one step of the computation of M on the (lexicographically) first string over the alphabet of M .

(Phase 2) Then it carries out two steps of the computation of M on each of the first *two* strings.

(Phase 3) Then it carries out three steps of the computation of M on each of the first three strings, and so on.

The first time M' discovers that M would accept a string, say w_1 , M' writes w_1 on its tape and pauses in state q to signal that $w_1 \in L$.

In general, after the i th string in L has been discovered, call it w_i , M' first displays it at state q , and starts all over from Phase 1, and so on, *keeping w_i on its tape*. Whenever it now discovers a string accepted by M , it first compares it with w_i ; if they are unequal, it continues. If it finds that the string just discovered is the same as w_i , then it looks for *the next* string accepted by M . This next string will be w_{i+1} . Again, w_{i+1} is displayed at state q , remembered, and compared. It is clear that any member of L will eventually be displayed.



4.6 Problems of Recursive Languages

The other direction is somewhat easier. If M enumerates L , then we can modify M to semidecide L as follows: Redesign M so that it saves any input supplied to it before beginning its enumeration process. Furthermore, every time M would enter the distinguished state q , the modified machine compares the current tape contents with the saved input string. If a match is found, the input string is accepted; otherwise, the enumeration process continues. The new machine then semidecides exactly the language enumerated by M . ■



4.6 Problems of Recursive Languages

Definition 5.7.2: Let M be a Turing machine enumerating a language L . We say that M **lexicographically enumerates** L if the following is true, where q is the special “display” state: Whenever $(q, \triangleright \sqcup w) \vdash_M^+ (q, \triangleright \sqcup w')$, then w' comes lexicographically after w . A language is **lexicographically Turing-enumerable** if and only if there is a Turing machine that lexicographically enumerates it.

Theorem 5.7.3: *A language is recursive if and only if it is lexicographically Turing-enumerable.*



4.6 Problems of Recursive Languages

Theorem 5.7.3: *A language is recursive if and only if it is lexicographically Turing-enumerable.*

Proof: Suppose that M is a Turing machine that decides L . Then the following Turing machine M' (which was our first unsuccessful attempt at proving the same direction of Theorem 5.7.2) lexicographically enumerates L : M' generates one after the other in lexicographic order all strings in the alphabet of L , and runs M on each. Whenever M accepts, M' displays the string and continues to the next one. If M rejects, M' simply continues to the next string without passing through the display state.

For the other direction, suppose that L is lexicographically enumerated by a Turing machine M . There are two cases: If L is *finite*, then there is nothing to prove, since in this case L is certainly recursive (as well as context-free, regular, etc.). So, suppose that L is infinite. The following machine M' decides L : On input w , start the enumerating machine M . Wait until either w is displayed, or any string that comes lexicographically after w is displayed. In the first case, accept w ; in the second, reject. Since there are finitely many strings that come lexicographically before w (fewer than $|\Sigma + 1|^{|w|}$) and L is infinite, we *know* that one of the two will happen. ■



Rice's Theorem

Rice's theorem. Let P be any nontrivial property of the language of a Turing machine. Prove that the problem of determining whether a given Turing machine's language has property P is undecidable.

In more formal terms, let P be a language consisting of Turing machine descriptions where P fulfills two conditions. First, P is nontrivial—it contains some, but not all, TM descriptions. Second, P is a property of the TM's language—whenever $L(M_1) = L(M_2)$, we have $\langle M_1 \rangle \in P$ iff $\langle M_2 \rangle \in P$. Here, M_1 and M_2 are any TMs. Prove that P is an undecidable language.

Assume for the sake of contradiction that P is a decidable language satisfying the properties and let R_P be a TM that decides P . We show how to decide A_{TM} using R_P by constructing TM S . First let $T_\emptyset$ be a TM that always rejects, so $L(T_\emptyset) = \emptyset$. You may assume that $\langle T_\emptyset \rangle \notin P$ without loss of generality, because you could proceed with $\bar{P}$ instead of P if $\langle T_\emptyset \rangle \in P$. Because P is not trivial, there exists a TM T with $\langle T \rangle \in P$. Design S to decide A_{TM} using R_P 's ability to distinguish between $T_\emptyset$ and T .

$S =$ "On input $\langle M, w \rangle$:

1. Use M and w to construct the following TM M_w .
 $M_w =$ "On input x :
 1. Simulate M on w . If it halts and rejects, *reject*.
If it accepts, proceed to stage 2.
 2. Simulate T on x . If it accepts, *accept*."
2. Use TM R_P to determine whether $\langle M_w \rangle \in P$. If YES, *accept*.
If NO, *reject*."

TM M_w simulates T if M accepts w . Hence $L(M_w)$ equals $L(T)$ if M accepts w and $\emptyset$ otherwise. Therefore $\langle M_w \rangle \in P$ iff M accepts w .

A_{TM} is reducible to P , A_{TM} is undecidable, so P is undecidable.



Theorem 5.7.4 (Rice's Theorem): Suppose that $\mathcal{C}$ is a proper, nonempty subset of the class of all recursively enumerable languages. Then the following problem is undecidable: Given a Turing machine M , is $L(M) \in \mathcal{C}$?

Proof: We can assume that $\emptyset \notin \mathcal{C}$ (otherwise, repeat the rest of the argument for the class of all recursively enumerable languages *not* in $\mathcal{C}$, which is also a proper, nonempty subset of the recursively enumerable languages). Next, since $\mathcal{C}$ is nonempty, we can assume that there is a language $L \in \mathcal{C}$ semidecided by machine M_L .

We shall reduce the halting problem to the problem of deciding whether the language semidecided by a given Turing machine is in $\mathcal{C}$. Suppose then that we are given a Turing machine M , and input w , and we wish to decide whether M halts on w . To accomplish this, we construct a Turing machine $T_{M,w}$, such that the language semidecided by $T_{M,w}$ is either the language L fixed above or the language $\emptyset$. On input x , $T_{M,w}$ simulates the universal Turing machine U on input " M " " w ". If it halts, then $T_{M,w}$, instead of halting, goes on to simulate M_L on input x : It either halts and accepts, or never halts, depending on the behavior of M_L on x . Naturally, if $U(\text{"M" "w"}) = \nearrow$, $T_{M,w}(x) = \nearrow$ as well. To review, $T_{M,w}$ is this machine:

$$T_{M,w}(x) : \text{if } U(\text{"M" "w"}) \neq \nearrow \text{ then } M_L(x) \text{ else } \nearrow$$



5.6 Problems of Recursive Languages

Claim: The language semidecided by $T_{M,w}$ is in class $\mathcal{C}$ if and only if M halts on input w .

Notice that the claim states that the construction of $T_{M,w}$ from M and w is a *reduction* of the halting problem to the problem of telling, given a Turing machine, whether the language semidecided by it is in $\mathcal{C}$. This would conclude the proof of the theorem

Proof of the claim: Suppose that M halts on input w . Then $T_{M,w}$ on input x determines this, and then always goes on to accept x if and only if $x \in L$. Hence, in this case, the language semidecided by $T_{M,w}$ is L —which is in $\mathcal{C}$.

Suppose then that $M(w) = \nearrow$. In this case $T_{M,w}$ *never* halts, and thus M_x semidecides the language $\emptyset$, known not to be in $\mathcal{C}$. ■

The undecidability of many problems follow from Rice's Theorem: Given a Turing machine M , is the language semidecided by it, $L(M)$, regular? context-free? finite? empty? Σ^* ? recursive? —and so on.



• Thanks !